MERN STACK

E-Commerce Web App

Roman Urdu Mein — Zero Se Hero Tak

CHAPTER 10 | FINAL CHAPTER | CHECKOUT + DEPLOY | ROMAN URDU

CHAPTER 10

Checkout, Orders, Payment & Deployment

ShopEase mukammal karo aur duniya ke saamne deploy karo!

Is Chapter Mein Aap Seekhenge:

- CheckoutScreen — shipping address form
- PaymentScreen — payment method select karna
- PlaceOrderScreen — final order summary
- Backend — order place karna API
- MyOrdersScreen — apne orders dekhna
- Admin OrdersScreen — saare orders manage karna
- Razorpay — payment gateway integrate karna
- Deployment — Render pe backend deploy karna
- Vercel pe React frontend deploy karna

■ ShopEase live — poori duniya use kar sake!

TABLE OF CONTENTS

- ⬜ 8.2 Account aur service setup
- ⬜ 8.3 Environment variables
- ⬜ 8.4 Deploy karna

9.0 Deployment — Frontend (Vercel)

- ⬜ 9.1 Vercel kya hai?
- ⬜ 9.2 Project import karna
- ⬜ 9.3 Environment variables
- ⬜ 9.4 Custom domain (optional)

10.0 Course Complete!

- ⬜ 10.1 Poore course ka recap
- ⬜ 10.2 Next steps — aage kya seekhein
- ⬜ 10.3 Portfolio tips

SECTION 1 — Checkout Flow Ka Plan

1.0 — 3-Step Checkout Process

ShopEase mein checkout 3 steps mein hoga — Amazon style! Har step ek alag page hai:

1	**Shipping — CheckoutScreen**	Address form — naam, street, city, zip
2	**Payment — PaymentScreen**	Method choose karo — Razorpay, COD
3	**Review — PlaceOrderScreen**	Summary dekho aur order confirm karo

■ TIP

Checkout data (address, payment method) Redux mein save karte hain — taake step 1 se step 3 tak data available rahe. Page refresh pe bhi localStorage se load hoga.

1.1 — orderSlice Banana

■ src/store/orderSlice.js

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import { createOrder, getMyOrders, getOrderById } from '../services/api';

// ■■ Async Thunks ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const placeOrder = createAsyncThunk(
  'orders/place',
  async (orderData, { rejectWithValue }) => {
    try {
      const { data } = await createOrder(orderData);
      return data.data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.message || 'Order fail!'
);
    }
  }
```

```
);

export const fetchMyOrders = createAsyncThunk(
  'orders/myOrders',
  async (_, { rejectWithValue }) => {
    try {
      const { data } = await getMyOrders();
      return data.data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.message || 'Error!');
    }
  }
);

// ■■ Slice ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const initialState = {
  // Checkout steps ka data
  shippingAddress: localStorage.getItem('shipping')
    ? JSON.parse(localStorage.getItem('shipping')) : {},
  paymentMethod: localStorage.getItem('paymentMethod') || 'Razorpay',

  // Order state
  currentOrder: null,
  myOrders:     [],
  loading:      false,
  error:        '',
};

const orderSlice = createSlice({
  name: 'orders',
  initialState,
  reducers: {
    // Step 1: Shipping address save karo
    saveShippingAddress: (state, action) => {
      state.shippingAddress = action.payload;
      localStorage.setItem('shipping', JSON.stringify(action.payload));
    },
```

```
    // Step 2: Payment method save karo
    savePaymentMethod: (state, action) => {
      state.paymentMethod = action.payload;
      localStorage.setItem('paymentMethod', action.payload);
    },
    clearCurrentOrder: (state) => { state.currentOrder = null; },
  },
  extraReducers: (builder) => {
    builder
      .addCase(placeOrder.pending,    s => { s.loading=true;  s.error=''; })
      .addCase(placeOrder.fulfilled,  (s,a) => { s.loading=false; s.currentOrder=a.payload; })
      .addCase(placeOrder.rejected,   (s,a) => { s.loading=false; s.error=a.payload; })
      .addCase(fetchMyOrders.pending,   s => { s.loading=true; })
      .addCase(fetchMyOrders.fulfilled, (s,a) => { s.loading=false; s.myOrders=a.payload; })
      .addCase(fetchMyOrders.rejected,  (s,a) => { s.loading=false; s.error=a.payload; });
  },
});

export const { saveShippingAddress, savePaymentMethod, clearCurrentOrder } = orderSlice.actions;
export const selectShipping     = s => s.orders.shippingAddress;
export const selectPaymentMethod= s => s.orders.paymentMethod;
export const selectCurrentOrder = s => s.orders.currentOrder;
export const selectMyOrders     = s => s.orders.myOrders;
export const selectOrderLoading = s => s.orders.loading;
export const selectOrderError   = s => s.orders.error;
export default orderSlice.reducer;
```

Store mein orderReducer add karo:

```
// src/store/store.js — orderReducer add karo
import orderReducer from './orderSlice';
```

```
const store = configureStore({
  reducer: {
    cart:    cartReducer,
    auth:    authReducer,
    products:productReducer,
    orders:  orderReducer,   // YEH ADD KARO
  },
});
```

1.2 — CheckoutScreen — Shipping Address Form

src/screens/CheckoutScreen.jsx

```
import { useState }              from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { useNavigate }           from 'react-router-dom';
import { saveShippingAddress,
         selectShipping }        from '../store/orderSlice';

function CheckoutScreen() {
  const dispatch  = useDispatch();
  const navigate  = useNavigate();
  const saved     = useSelector(selectShipping);

  // Pehle se saved address? Toh woh load karo
  const [address,  setAddress]  = useState(saved.address  || '');
  const [city,     setCity]     = useState(saved.city     || '');
  const [state,    setState]    = useState(saved.state    || '');
  const [zipCode,  setZipCode]  = useState(saved.zipCode  || '');
  const [phone,    setPhone]    = useState(saved.phone    || '');

  const submitHandler = (e) => {
    e.preventDefault();
    // Redux mein save karo
    dispatch(saveShippingAddress({ address, city, state, zipCode, phone }
));
    // Step 2 pe jao
    navigate('/payment');
```

```
  };

  const inputStyle = {
    width:'100%', padding:'10px 14px', marginBottom:'14px',
    background:'#1C2128', border:'1px solid #30363D',
    borderRadius:'6px', color:'white', fontSize:'14px', outline:'none',
  };

  return (
    <div style={{ maxWidth:'500px', margin:'40px auto', padding:'0 20px'
}}>
      {/* Progress bar */}
      <div style={{ display:'flex', gap:'8px', marginBottom:'32px' }}>
        {['Shipping','Payment','Review'].map((step,i) => (
          <div key={step} style={{ flex:1, textAlign:'center',
            padding:'8px', borderRadius:'6px', fontSize:'13px',
            background: i===0 ? '#34D399' : '#1C2128',
            color: i===0 ? '#0D1117' : '#8B949E', fontWeight: i===0 ? 'bo
ld':'normal' }}>
            {i+1}. {step}
          </div>
        ))}
      </div>

      <h2 style={{ color:'#E6EDF3', marginBottom:'20px' }}>Shipping Addre
ss</h2>

      <form onSubmit={submitHandler}>
        <label style={{ color:'#8B949E', fontSize:'13px' }}>Ghar ka Addre
ss</label>
        <input style={inputStyle} value={address}
          onChange={e=>setAddress(e.target.value)} required
          placeholder='Street, Mohalla, Area' />

        <label style={{ color:'#8B949E', fontSize:'13px' }}>Shehar</label
>
        <input style={inputStyle} value={city}
          onChange={e=>setCity(e.target.value)} required placeholder='Lah
ore' />
```

```
        <label style={{ color:'#8B949E', fontSize:'13px' }}>State/Province</label>
        <input style={inputStyle} value={state}
          onChange={e=>setState(e.target.value)} required placeholder='Punjab' />
        <label style={{ color:'#8B949E', fontSize:'13px' }}>Zip / Postal Code</label>
        <input style={inputStyle} value={zipCode}
          onChange={e=>setZipCode(e.target.value)} required placeholder='54000' />
        <label style={{ color:'#8B949E', fontSize:'13px' }}>Phone Number</label>
        <input style={inputStyle} value={phone}
          onChange={e=>setPhone(e.target.value)} required placeholder='03001234567' />
        <button type='submit' style={{
          width:'100%', padding:'14px', background:'#34D399',
          color:'#0D1117', border:'none', borderRadius:'8px',
          fontSize:'16px', fontWeight:'bold', cursor:'pointer' }}>
          Aage Chalein &rarr;
        </button>
      </form>
    </div>
  );
}
export default CheckoutScreen;
```

SECTION 2 — PaymentScreen

■ src/screens/PaymentScreen.jsx

```
import { useState }              from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { useNavigate }           from 'react-router-dom';
import { savePaymentMethod,
         selectPaymentMethod }   from '../store/orderSlice';

const PAYMENT_METHODS = [
  { id: 'Razorpay', label: 'Razorpay (Card / UPI / Net Banking)', icon: '
■' },
  { id: 'COD',      label: 'Cash on Delivery',                   icon: '
■' },
];

function PaymentScreen() {
  const dispatch = useDispatch();
  const navigate = useNavigate();
  const saved    = useSelector(selectPaymentMethod);
  const [method, setMethod] = useState(saved || 'Razorpay');

  const submitHandler = (e) => {
    e.preventDefault();
    dispatch(savePaymentMethod(method));
    navigate('/placeorder');
  };

  return (
    <div style={{ maxWidth:'500px', margin:'40px auto', padding:'0 20px'
}}>
      {/* Progress bar */}
      <div style={{ display:'flex', gap:'8px', marginBottom:'32px' }}>
        {['Shipping','Payment','Review'].map((step,i) => (
          <div key={step} style={{ flex:1, textAlign:'center',
            padding:'8px', borderRadius:'6px', fontSize:'13px',
```

```
            background: i===1 ? '#34D399' : '#1C2128',
            color: i===1 ? '#0D1117' : '#8B949E',
            fontWeight: i===1 ? 'bold':'normal' }}>
            {i+1}. {step}
          </div>
        ))}
      </div>

      <h2 style={{ color:'#E6EDF3', marginBottom:'20px' }}>Payment Method
</h2>

      <form onSubmit={submitHandler}>
        {PAYMENT_METHODS.map(pm => (
          <label key={pm.id} style={{
            display:'flex', alignItems:'center', gap:'12px',
            padding:'16px', marginBottom:'12px',
            background: method===pm.id ? '#0A2818' : '#1C2128',
            border: `1px solid ${method===pm.id ? '#34D399' : '#30363D'}`
,
            borderRadius:'8px', cursor:'pointer',
          }}>
            <input type='radio' name='payment' value={pm.id}
              checked={method===pm.id}
              onChange={e => setMethod(e.target.value)}
              style={{ accentColor:'#34D399', width:'18px', height:'18px'
 }} />
            <span style={{ fontSize:'20px' }}>{pm.icon}</span>
            <span style={{ color:'#E6EDF3', fontSize:'14px' }}>{pm.label}
</span>
          </label>
        ))}

        <button type='submit' style={{
          width:'100%', padding:'14px', background:'#34D399',
          color:'#0D1117', border:'none', borderRadius:'8px',
          fontSize:'16px', fontWeight:'bold', cursor:'pointer',
          marginTop:'8px' }}>
```

```
            Review Order &rarr;
          </button>
        </form>
      </div>
    );
  }
  export default PaymentScreen;
```

SECTION 3 — PlaceOrderScreen

3.0 — Final Review aur Order Place Karna

■ src/screens/PlaceOrderScreen.jsx

```
import { useEffect }              from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { useNavigate, Link }       from 'react-router-dom';
import { placeOrder, clearCurrentOrder,
         selectShipping, selectPaymentMethod,
         selectCurrentOrder, selectOrderLoading,
         selectOrderError }        from '../store/orderSlice';
import { clearCart, selectCartItems,
         selectTotalPrice }        from '../store/cartSlice';

function PlaceOrderScreen() {
  const dispatch       = useDispatch();
  const navigate       = useNavigate();
  const cartItems      = useSelector(selectCartItems);
  const totalPrice     = useSelector(selectTotalPrice);
  const shippingAddress= useSelector(selectShipping);
  const paymentMethod  = useSelector(selectPaymentMethod);
  const currentOrder   = useSelector(selectCurrentOrder);
  const loading        = useSelector(selectOrderLoading);
  const error          = useSelector(selectOrderError);

  // Order place hone ke baad — OrderDetail page pe jao
  useEffect(() => {
    if (currentOrder) {
      dispatch(clearCart());      // Cart khaali karo
      navigate(`/orders/${currentOrder._id}`);
      dispatch(clearCurrentOrder());
    }
  }, [currentOrder, navigate, dispatch]);
```

```
  // Calculated prices
  const taxPrice      = totalPrice * 0.05;
  const shippingPrice = totalPrice > 2000 ? 0 : 150;
  const grandTotal    = totalPrice + taxPrice + shippingPrice;

  const placeOrderHandler = () => {
    dispatch(placeOrder({
      orderItems: cartItems.map(i => ({
        product:  i._id,
        name:     i.name,
        image:    i.image,
        price:    i.price,
        quantity: i.quantity,
      })),
      shippingAddress,
      paymentMethod,
      itemsPrice:    totalPrice,
      taxPrice:      taxPrice,
      shippingPrice: shippingPrice,
      totalPrice:    grandTotal,
    }));
  };

  return (
    <div style={{ maxWidth:'1000px', margin:'40px auto', padding:'0 20px'
 }}>
      {/* Progress bar */}
      <div style={{ display:'flex', gap:'8px', marginBottom:'32px' }}>
        {['Shipping','Payment','Review'].map((step,i) => (
          <div key={step} style={{ flex:1, textAlign:'center', padding:'8
px',
            borderRadius:'6px', fontSize:'13px',
            background: i===2 ? '#34D399' : '#1C2128',
            color: i===2 ? '#0D1117' : '#8B949E',
            fontWeight: i===2 ? 'bold':'normal' }}>
            {i+1}. {step}
```

```
          </div>
        ))}
      </div>

      <div style={{ display:'grid', gridTemplateColumns:'3fr 2fr', gap:'24px' }}>
        {/* LEFT — Order Details */}
        <div>
          {/* Shipping Info */}
          <div style={{ background:'#161B22', border:'1px solid #30363D',
            borderRadius:'10px', padding:'16px', marginBottom:'16px' }}>
            <h3 style={{ color:'#E6EDF3', marginBottom:'8px' }}>Shipping Address</h3>
            <p style={{ color:'#8B949E' }}>
              {shippingAddress.address}, {shippingAddress.city},<br/>
              {shippingAddress.state} — {shippingAddress.zipCode}<br/>
              ■ {shippingAddress.phone}
            </p>
            <Link to='/checkout' style={{ color:'#34D399', fontSize:'13px' }}>
              Badlna hai? Edit karo
            </Link>
          </div>

          {/* Payment Method */}
          <div style={{ background:'#161B22', border:'1px solid #30363D',
            borderRadius:'10px', padding:'16px', marginBottom:'16px' }}>
            <h3 style={{ color:'#E6EDF3', marginBottom:'8px' }}>Payment Method</h3>
            <p style={{ color:'#8B949E' }}>
              {paymentMethod === 'COD' ? '■ Cash on Delivery' : '■ Razorpay'}
            </p>
            <Link to='/payment' style={{ color:'#34D399', fontSize:'13px' }}>Edit</Link>
          </div>

          {/* Order Items */}
```

```
          <div style={{ background:'#161B22', border:'1px solid #30363D',
            borderRadius:'10px', padding:'16px' }}>
            <h3 style={{ color:'#E6EDF3', marginBottom:'12px' }}>Order It
ems</h3>
            {cartItems.map(item => (
              <div key={item._id} style={{ display:'flex', gap:'12px',
                alignItems:'center', marginBottom:'10px' }}>
                <img src={item.image} alt={item.name}
                  style={{ width:'50px', height:'50px', objectFit:'cover'
, borderRadius:'6px' }} />
                <Link to={`/product/${item._id}`}
                  style={{ color:'#E6EDF3', flex:1, textDecoration:'none'
, fontSize:'14px' }}>
                  {item.name}
                </Link>
                <span style={{ color:'#8B949E', fontSize:'13px' }}>
                  {item.quantity} x Rs.{item.price?.toLocaleString()}
                  = Rs.{(item.quantity*item.price).toLocaleString()}
                </span>
              </div>
            ))}
          </div>
        </div>

        {/* RIGHT — Order Summary + Place Button */}
        <div style={{ background:'#161B22', border:'1px solid #30363D',
          borderRadius:'10px', padding:'20px', height:'fit-content' }}>
          <h3 style={{ color:'#E6EDF3', marginBottom:'16px' }}>Order Summ
ary</h3>
          {[
            ['Items', totalPrice],
            ['Tax (5%)', taxPrice],
            ['Shipping', shippingPrice],
          ].map(([label,amount]) => (
            <div key={label} style={{ display:'flex', justifyContent:'spa
ce-between',
```

```
              marginBottom:'10px' }}>
              <span style={{ color:'#8B949E' }}>{label}</span>
              <span style={{ color:'#E6EDF3' }}>Rs. {Math.round(amount).t
oLocaleString()}</span>
            </div>
          ))}
          <hr style={{ borderColor:'#30363D', margin:'12px 0' }} />
          <div style={{ display:'flex', justifyContent:'space-between', m
arginBottom:'20px' }}>
            <span style={{ color:'white', fontWeight:'bold' }}>Total</spa
n>
            <span style={{ color:'#34D399', fontWeight:'bold', fontSize:'
18px' }}>
              Rs. {Math.round(grandTotal).toLocaleString()}
            </span>
          </div>
          {error && <p style={{ color:'#FF6B6B', marginBottom:'12px' }}>{
error}</p>}
          <button onClick={placeOrderHandler} disabled={loading || cartIt
ems.length===0}
            style={{ width:'100%', padding:'14px', background:'#34D399',
              color:'#0D1117', border:'none', borderRadius:'8px',
              fontSize:'16px', fontWeight:'bold',
              cursor: loading ? 'wait' : 'pointer' }}>
            {loading ? 'Order Place Ho Raha Hai...' : 'Order Place Karo'}
          </button>
        </div>
      </div>
    </div>
  );
}
export default PlaceOrderScreen;
```

SECTION 4 — MyOrdersScreen aur Routes Update

4.0 — MyOrdersScreen

■ src/screens/MyOrdersScreen.jsx

```
import { useEffect }              from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { Link }                   from 'react-router-dom';
import { fetchMyOrders, selectMyOrders,
         selectOrderLoading }      from '../store/orderSlice';

// Status ke hisaab se badge color
const statusColor = {
  Processing: '#FFD93D',
  Shipped:    '#82AAFF',
  Delivered:  '#56CF8A',
  Cancelled:  '#FF6B6B',
};

function MyOrdersScreen() {
  const dispatch = useDispatch();
  const orders   = useSelector(selectMyOrders);
  const loading  = useSelector(selectOrderLoading);

  useEffect(() => { dispatch(fetchMyOrders()); }, [dispatch]);

  if (loading) return <p style={{color:'#34D399',textAlign:'center',margi
nTop:'40px'}}>Loading...</p>;

  if (orders.length === 0) return (
    <div style={{ textAlign:'center', marginTop:'60px' }}>
      <p style={{ fontSize:'50px' }}>■</p>
      <h2 style={{ color:'#E6EDF3' }}>Koi order nahi abhi tak</h2>
      <Link to='/' style={{ color:'#34D399' }}>Shopping karo!</Link>
    </div>
```

```
  );

  return (
    <div style={{ maxWidth:'900px', margin:'40px auto', padding:'0 20px'
}}>
      <h1 style={{ color:'#E6EDF3', marginBottom:'24px' }}>Mere Orders</h
1>
      <div style={{ overflowX:'auto' }}>
        <table style={{ width:'100%', borderCollapse:'collapse' }}>
          <thead>
            <tr style={{ background:'#1C2128' }}>
              {['Order ID','Date','Total','Payment','Status',''].map(h =>
 (
                <th key={h} style={{ padding:'12px', color:'#8B949E',
                  textAlign:'left', borderBottom:'1px solid #30363D',
                  fontSize:'13px' }}>{h}</th>
              ))}
            </tr>
          </thead>
          <tbody>
            {orders.map(order => (
              <tr key={order._id} style={{ borderBottom:'1px solid #30363
D' }}>
                <td style={{ padding:'12px', color:'#8B949E', fontSize:'1
2px' }}>
                  {order._id.slice(-8).toUpperCase()}
                </td>
                <td style={{ padding:'12px', color:'#E6EDF3', fontSize:'1
3px' }}>
                  {new Date(order.createdAt).toLocaleDateString('ur-PK')}
                </td>
                <td style={{ padding:'12px', color:'#56CF8A', fontWeight:
'bold' }}>
                  Rs. {order.totalPrice?.toLocaleString()}
                </td>
                <td style={{ padding:'12px', fontSize:'13px',
                  color: order.isPaid ? '#56CF8A' : '#FF6B6B' }}>
```

```
                  {order.isPaid ? 'Paid' : 'Unpaid'}
                </td>
                <td style={{ padding:'12px' }}>
                  <span style={{
                    padding:'4px 10px', borderRadius:'12px', fontSize:'12
px',
                    background: statusColor[order.orderStatus] + '22',
                    color: statusColor[order.orderStatus] || '#8B949E',
                    fontWeight:'bold'
                  }}>
                    {order.orderStatus}
                  </span>
                </td>
                <td style={{ padding:'12px' }}>
                  <Link to={`/orders/${order._id}`}
                    style={{ color:'#34D399', fontSize:'13px', textDecora
tion:'none' }}>
                    Details &rarr;
                  </Link>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}
export default MyOrdersScreen;
```

4.1 — App.js — Naye Routes Add Karo

■ src/App.js — Updated Routes

```
// src/App.js — yeh saari imports aur routes add karo
import CheckoutScreen    from './screens/CheckoutScreen';
```

```
import PaymentScreen     from './screens/PaymentScreen';
import PlaceOrderScreen  from './screens/PlaceOrderScreen';
import MyOrdersScreen    from './screens/MyOrdersScreen';
import AdminOrdersScreen from './screens/AdminOrdersScreen';

// Routes mein add karo:
<Route path='/checkout'   element={<ProtectedRoute><CheckoutScreen /></Pr
otectedRoute>} />
<Route path='/payment'    element={<ProtectedRoute><PaymentScreen /></Pro
tectedRoute>} />
<Route path='/placeorder' element={<ProtectedRoute><PlaceOrderScreen /></
ProtectedRoute>} />
<Route path='/myorders'   element={<ProtectedRoute><MyOrdersScreen /></Pr
otectedRoute>} />
<Route path='/orders/:id' element={<ProtectedRoute><OrderDetailScreen /><
/ProtectedRoute>} />

// Admin routes:
<Route path='/admin/orders'
  element={<AdminRoute><AdminOrdersScreen /></AdminRoute>} />
```

SECTION 5 — Admin Panel

5.0 — AdminOrdersScreen

■ src/screens/AdminOrdersScreen.jsx

```
import { useEffect, useState }    from 'react';
import { useSelector }            from 'react-redux';
import { selectToken }            from '../store/authSlice';
import { getAllOrders, updateOrderStatus } from '../services/api';

const STATUS_OPTIONS = ['Processing','Shipped','Delivered','Cancelled'];
const statusColor    = { Processing:'#FFD93D', Shipped:'#82AAFF',
                         Delivered:'#56CF8A', Cancelled:'#FF6B6B' };

function AdminOrdersScreen() {
  const [orders,  setOrders]  = useState([]);
  const [loading, setLoading] = useState(true);
  const token = useSelector(selectToken);

  const fetchOrders = async () => {
    try {
      const { data } = await getAllOrders();
      setOrders(data.data);
    } catch (err) { console.error(err); }
    finally { setLoading(false); }
  };

  useEffect(() => { fetchOrders(); }, []);

  const updateStatus = async (orderId, newStatus) => {
    try {
      await updateOrderStatus(orderId, newStatus);
      // Local state update karo
      setOrders(prev => prev.map(o =>
        o._id === orderId ? { ...o, orderStatus: newStatus } : o
      ));
```

```
    } catch (err) { alert('Update fail hua!'); }
  };

  if (loading) return <p style={{ color:'#C084FC', textAlign:'center' }}>
Loading...</p>;

  return (
    <div style={{ maxWidth:'1100px', margin:'30px auto', padding:'0 20px'
 }}>
      <h1 style={{ color:'#E6EDF3', marginBottom:'20px' }}>
        Admin — Orders ({orders.length})
      </h1>
      <div style={{ overflowX:'auto' }}>
        <table style={{ width:'100%', borderCollapse:'collapse' }}>
          <thead>
            <tr style={{ background:'#1C2128' }}>
              {['ID','User','Date','Total','Status','Action'].map(h => (
                <th key={h} style={{ padding:'12px', color:'#8B949E',
                  textAlign:'left', borderBottom:'1px solid #30363D' }}>{
h}</th>
              ))}
            </tr>
          </thead>
          <tbody>
            {orders.map(order => (
              <tr key={order._id} style={{ borderBottom:'1px solid #30363
D' }}>
                <td style={{ padding:'10px', color:'#8B949E', fontSize:'1
2px' }}>
                  {order._id.slice(-6).toUpperCase()}
                </td>
                <td style={{ padding:'10px', color:'#E6EDF3', fontSize:'1
3px' }}>
                  {order.user?.name || 'N/A'}
                </td>
                <td style={{ padding:'10px', color:'#8B949E', fontSize:'1
2px' }}>
```

```
                  {new Date(order.createdAt).toLocaleDateString()}
                </td>
                <td style={{ padding:'10px', color:'#56CF8A', fontWeight:
'bold' }}>
                  Rs. {order.totalPrice?.toLocaleString()}
                </td>
                <td style={{ padding:'10px' }}>
                  <span style={{ padding:'3px 8px', borderRadius:'10px',
fontSize:'12px',
                    background: statusColor[order.orderStatus]+'22',
                    color: statusColor[order.orderStatus] }}>
                    {order.orderStatus}
                  </span>
                </td>
                <td style={{ padding:'10px' }}>
                  {/* Status dropdown */}
                  <select
                    value={order.orderStatus}
                    onChange={e => updateStatus(order._id, e.target.value
)}
                    style={{ background:'#1C2128', color:'white',
                      border:'1px solid #30363D', borderRadius:'4px',
                      padding:'4px 8px', fontSize:'12px' }}
                  >
                    {STATUS_OPTIONS.map(s => (
                      <option key={s} value={s}>{s}</option>
                    ))}
                  </select>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
```

```
  );
}
export default AdminOrdersScreen;
```

5.1 — api.js Mein Order Functions Add Karo

■ src/services/api.js — Order functions

```
// Yeh functions api.js mein add karo:
export const createOrder       = (data)         => API.post('/orders', da
ta);
export const getMyOrders       = ()              => API.get('/orders/myor
ders');
export const getOrderById      = (id)            => API.get(`/orders/${id
}`);
export const getAllOrders       = ()              => API.get('/orders');
export const updateOrderStatus = (id, status)   =>
  API.put(`/orders/${id}/status`, { orderStatus: status });
```

SECTION 6 — Razorpay Payment Gateway

6.0 — Razorpay Kya Hai?

Razorpay India ka popular payment gateway hai. Card, UPI, Net Banking, Wallets — sab support karta hai. Free account mein test mode milta hai — real paise nahi lagte practice mein!

Feature	COD (Cash on Delivery)	Razorpay
Payment	Delivery pe cash	Online — instant
Risk	Return rate zyada	Pehle payment — safe
Setup	Koi setup nahi	API keys + frontend code
Test mode	N/A	Test cards available — koi real money nahi

6.1 — Razorpay Setup

1. razorpay.com pe account banao — Free plan choose karo
2. Dashboard: Settings -> API Keys -> Generate Test Key
3. KEY_ID aur KEY_SECRET copy karo
4. .env mein add karo:

```
# Backend .env mein add karo:
RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxx
RAZORPAY_KEY_SECRET=xxxxxxxxxxxxxxx

# Frontend .env mein add karo:
REACT_APP_RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxx
```

6.2 — Backend — Razorpay Order Create

```
# Backend mein install karo:
npm install razorpay
```

```
// controllers/orderController.js mein add karo:
const Razorpay = require('razorpay');

const razorpay = new Razorpay({
```

```
  key_id:     process.env.RAZORPAY_KEY_ID,
  key_secret: process.env.RAZORPAY_KEY_SECRET,
});

// @route  POST /api/orders/:id/razorpay
// @access Private
const createRazorpayOrder = asyncHandler(async (req, res) => {
  const order = await Order.findById(req.params.id);
  if (!order) { res.status(404); throw new Error('Order nahi mila!'); }

  // Razorpay order banao (amount paisa mein — Rs. x 100)
  const rzpOrder = await razorpay.orders.create({
    amount:   Math.round(order.totalPrice * 100),
    currency: 'INR',
    receipt:  `order_${order._id}`,
  });

  res.json({
    orderId:  rzpOrder.id,
    amount:   rzpOrder.amount,
    currency: rzpOrder.currency,
    keyId:    process.env.RAZORPAY_KEY_ID,
  });
});

// @route  POST /api/orders/:id/verify
// @access Private
const verifyPayment = asyncHandler(async (req, res) => {
  const crypto = require('crypto');
  const { razorpay_order_id, razorpay_payment_id, razorpay_signature } =
req.body;

  // Signature verify karo
  const body    = razorpay_order_id + '|' + razorpay_payment_id;
  const expected= crypto
    .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
    .update(body)
    .digest('hex');
```

```
  if (expected !== razorpay_signature) {
    res.status(400); throw new Error('Payment verify nahi hua!');
  }

  // Order paid mark karo
  const order       = await Order.findById(req.params.id);
  order.isPaid      = true;
  order.paidAt      = Date.now();
  order.orderStatus = 'Processing';
  order.paymentResult = { id: razorpay_payment_id, status: 'paid' };
  await order.save();

  res.json({ success: true, message: 'Payment successful!' });
});

// Routes add karo (orderRoutes.js mein):
router.post('/:id/razorpay', protect, createRazorpayOrder);
router.post('/:id/verify',   protect, verifyPayment);
```

6.3 — Frontend — Razorpay Button

```
// OrderDetailScreen.jsx mein — payment button
// index.html mein Razorpay script add karo:
// &lt;script src='https://checkout.razorpay.com/v1/checkout.js'&gt;&lt;/script&gt;

const payWithRazorpay = async () => {
  try {
    // Backend se Razorpay order create karo
    const { data } = await API.post(`/orders/${order._id}/razorpay`);

    // Razorpay checkout open karo
    const options = {
      key:         data.keyId,
      amount:      data.amount,
      currency:    data.currency,
      order_id:    data.orderId,
      name:        'ShopEase',
```

```
      description: `Order #${order._id.slice(-6).toUpperCase()}`,
      handler: async (response) => {
        // Payment successful — backend ko verify karo
        await API.post(`/orders/${order._id}/verify`, response);
        alert('Payment ho gayi! Order confirmed!');
        window.location.reload();
      },
      prefill: {
        name:  user?.name,
        email: user?.email,
        contact: order.shippingAddress?.phone,
      },
      theme: { color: '#34D399' },
    };
    const rzp = new window.Razorpay(options);
    rzp.open();
  } catch (err) {
    alert('Payment start nahi hua: ' + err.message);
  }
};

// Button:
{!order.isPaid && order.paymentMethod === 'Razorpay' && (
  <button onClick={payWithRazorpay} style={{
    background:'#34D399', color:'#0D1117', padding:'12px 24px',
    border:'none', borderRadius:'8px', fontWeight:'bold', cursor:'pointer
' }}>
    Razorpay Se Pay Karo
  </button>
)}
```

■ TIP

Test mode mein Razorpay test card use karo: Card: 4111 1111 1111 1111 | Expiry: any future date | CVV: any 3 digits. Real paise nahi katenge — sirf test hoga!

SECTION 7 — Deployment — Backend (Render)

7.0 — Render Kya Hai?

Render ek free cloud hosting platform hai jahan Node.js apps deploy kar sakte ho. Free tier mein backend host hota hai — initially thoda slow wake-up hota hai, lekin practice ke liye bilkul theek hai!

7.1 — Pehle Yeh Prepare Karo

1. GitHub pe code push karo

```
# Shopease root folder mein (backend wala)
git init
git add .
git commit -m "ShopEase backend ready for deployment"

# GitHub pe naya repo banao — phir:
git remote add origin https://github.com/tumhara-naam/shopease-backend.git
git push -u origin main
```

2. package.json mein start script check karo

```
// package.json mein yeh hona chahiye:
"scripts": {
  "start": "node server.js",
  "dev":   "nodemon server.js"
}

// Render 'npm start' use karta hai production mein
```

7.2 — Render Pe Deploy Karna (Step by Step)

01	**render.com pe jao**	Browser mein render.com kholao — GitHub se sign up karo
02	**New + -> Web Service**	Dashboard pe 'New +' click karo -> 'Web Service' choose karo

03	**GitHub repo connect karo**	shopease-backend repo select karo
04	**Settings fill karo**	Name: shopease-api | Region: Singapore | Branch: main
05	**Build Command**	npm install
06	**Start Command**	npm start
07	**Environment Variables**	Add karo: MONGO_URI, JWT_SECRET, PORT, NODE_ENV=production, CLIENT_URL
08	**Create Web Service**	Deploy shuru! 3-5 minute lagenge
09	**URL copy karo**	https://shopease-api.onrender.com — yahi tumhara backend URL hai

■ YAAD RAHO

Render free tier mein server 15 minute inactivity ke baad 'sleep' ho jaata hai. Pehli request pe 30-60 second lag sakte hain — yeh normal hai. Paid plan mein yeh masla nahi hota.

SECTION 8 — Deployment — Frontend (Vercel)

8.0 — Frontend Deploy Karna

Vercel React apps ke liye sabse asaan platform hai. GitHub se connect karo, ek click mein deploy ho jaata hai, aur free HTTPS domain milta hai!

8.1 — Pehle api.js Update Karo

Frontend mein backend ka URL localhost se Render URL par change karo:

■ src/services/api.js — Production URL

```
// api.js mein baseURL update karo:
const API = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'http://localhost:5000/api',
});

// frontend/.env.local mein (development ke liye):
REACT_APP_API_URL=http://localhost:5000/api

// Vercel pe production mein env variable add karoge:
REACT_APP_API_URL=https://shopease-api.onrender.com/api
```

8.2 — Frontend GitHub Pe Push Karo

```
# frontend folder mein jao
cd frontend

# GitHub pe alag repo banao — shopease-frontend
git init
git add .
git commit -m "ShopEase frontend ready"
git remote add origin https://github.com/tumhara-naam/shopease-frontend.git
git push -u origin main
```

8.3 — Vercel Pe Deploy (Step by Step)

01	**vercel.com pe jao**	vercel.com kholao — GitHub se sign up karo
02	**Add New -> Project**	Dashboard pe 'Add New' -> 'Project' click karo
03	**GitHub repo import karo**	shopease-frontend repo select karo
04	**Framework: React**	Vercel automatically detect karega — Create React App ya Vite
05	**Environment Variables**	REACT_APP_API_URL = https://shopease-api.onrender.com/api
06	**REACT_APP_RAZORPAY_KEY_ID**	rzp_test_xxxxxxxxxx (Razorpay key)
07	**Deploy click karo**	1-2 minute mein ready! URL milega
08	**URL share karo!**	https://shopease.vercel.app — duniya dekh sakti hai!

8.4 — Backend CORS Update Karo

Render pe backend deploy hone ke baad, Vercel URL ko CORS mein allow karo:

```
// server.js mein CORS update karo:
app.use(cors({
  origin: [
    'http://localhost:3000',            // Development
    'https://shopease.vercel.app',      // Production
    process.env.CLIENT_URL,             // .env se bhi
  ].filter(Boolean),
  credentials: true,
}));

// Render pe CLIENT_URL env variable add karo:
CLIENT_URL=https://shopease.vercel.app
```

■ SAHI KIYA

Code update karne ke baad sirf 'git push' karo — Render aur Vercel automatically redeploy kar denge! Ek baar setup ke baad yeh bohat convenient hai.

SECTION 9 — Course Complete! Recap aur Next Steps

9.0 — Poore Course Ka Recap

10 chapters mein tumne ek poori E-Commerce app banai — bilkul zero se leke live deployment tak! Yeh sab tumne seekha:

Chapter	Topic	Key Concepts
Ch 1	Introduction & Setup	Node.js, npm, VS Code, package.json, variables, data types
Ch 2	Express.js & HTTP	Server banana, routes, GET/POST/PUT/DELETE, middleware, CORS
Ch 3	MongoDB & Mongoose	NoSQL, Atlas setup, Schema, Model, CRUD, queries, populate
Ch 4	MVC Architecture	Controllers, folder structure, asyncHandler, error middleware
Ch 5	Authentication & Authorization	bcrypt, JWT, protect middleware, admin middleware
Ch 6	React.js Basics	JSX, Components, Props, useState, useEffect, Axios
Ch 7	React Router & Hooks	Routing, useParams, useNavigate, Context, useReducer, Custom Hooks
Ch 8	Redux Toolkit	Store, Slices, useSelector, useDispatch, createAsyncThunk
Ch 9	Frontend Pages	HomeScreen, ProductScreen, CartScreen, Search, Filter, Pagination
Ch 10	Checkout, Orders & Deploy	Checkout flow, Razorpay, Render + Vercel deployment

9.1 — Aage Kya Seekhein?

TypeScript	JavaScript ko type-safe banana — industry standard
Next.js	React + SSR + file-based routing — SEO friendly
Docker	App ko container mein band karna — anywhere run karo
Testing	Jest + React Testing Library — bugs pehle pakdo

GraphQL	REST ka alternative — flexible API queries
Socket.io	Real-time features — live chat, notifications
AWS / GCP	Enterprise cloud — S3, EC2, Lambda
CI/CD	GitHub Actions — auto test aur deploy
Redis	Caching — app ko faster banana
Microservices	Badi apps ko chhote services mein todna

9.2 — Portfolio Tips

1. GitHub pe ShopEase ka code rakho — aur README achhi tarah likho (project kya hai, screenshots, run kaise karein)
2. Live URL zaroor rakho — Render + Vercel free hain — interviewer directly dekhna chahega
3. Admin account bana ke kuch products add karo — demo ready raho hamesha
4. Mobile responsive banana — CSS Grid aur media queries add karo
5. README mein technologies list karo — MongoDB, Express, React, Node.js, Redux, JWT, Razorpay
6. Next project shuru karo — ek project se portfolio nahi banta — 2-3 hone chahiyen

MUBARAK HO! ShopEase Complete Hua! ■■

Tumne 10 chapters mein poora MERN stack seekha — MongoDB se React tak, JWT se Razorpay tak, aur local machine se live deployment tak. Yeh wahi skills hain jo companies hire karte waqt dekhti hain. Ab yeh code GitHub pe daalo, live link portfolio mein add karo, aur agla project shuru karo.

Tum developer ban gaye ho — bilkul sach mein! ■